HADES

Database Security — Complete Manual
The db_scan module explained for beginners

For authorised security testing only.
Scanning or exploiting a database without written permission is illegal. This manual explains what the Database Security module checks, how to run it, how to read its output, and how to remediate the issues it finds.

Generated 2026-06-07

How to read this manual

This manual walks through the db_scan profile from the ground up: what it does, how to run it, every check it performs, and how to act on the results. Pale-pink bars are sub-headings. No prior knowledge is required.

1 · Introduction — What is a database?

A database is a website's 'vault': it stores accounts, passwords, orders, messages — all the data. If someone reaches it without authorisation, that is a major, often irreversible, data breach.

The engines Hades knows

Hades knows MySQL / MariaDB, PostgreSQL, Microsoft SQL Server, Oracle, MongoDB, Redis, Elasticsearch, CouchDB, Cassandra and Memcached. Some 'speak' the SQL language (MySQL, PostgreSQL); others are called 'NoSQL' (MongoDB, Redis) and use different formats.

No prior knowledge required

This manual assumes no security background. Every technical term is explained as it appears, and there is a glossary at the end.

2 · Why audit database security

A database exposed to the Internet is one of the most common causes of massive leaks: millions of customer records have been stolen from MongoDB or Elasticsearch databases left open with no password at all. Exposed Redis servers have been used to deploy ransomware.

What an attacker looks for

1) a database port reachable from the Internet; 2) the absence of a password; 3) credentials written in plain text in a file or the site's code; 4) a SQL injection that lets them read the database through the website itself. The db_scan module tests exactly these four things — and more.

3 · The Hades db_scan module

db_scan is a dedicated scan profile: a single specialised module (`scanner/db/db_security.py`) that performs a complete audit of a target's 'database' surface, end to end, then helps you exploit what it finds.

In one sentence

It scans database ports, tests password-free access (and extracts data to prove it), hunts for SQL / NoSQL injection, tracks down leaked secret and credential files, then computes an exposure score, an attack plan, and a summary of the 'loot' collected.

Detection first, exploitation second

By default, Hades only DETECTS. Real exploitation (launching the actual sqlmap tool) happens only if you add the `--exploit` option and confirm you are authorised. Nothing is ever exploited automatically.

4 · Installation and prerequisites

Requirements: Python 3.10 or newer. Install the dependencies with `pip install -r requirements.txt`. That is all you need for detection.

Optional: sqlmap

For SQL-injection exploitation (`--exploit`), install sqlmap: `pip install sqlmap`. Hades finds it automatically, even if it is not on the PATH (it looks in Python's Scripts folder).

5 · Usage — the commands

Basic audit

```
python hades.py --url https://target.com --profile db_scan
```

Runs the full database audit and prints the result in the terminal.

Reports (always written)

Every scan automatically writes a styled HTML report (dark theme, with the attack path and the loot) and a structured JSON report to `reports/`, and opens the HTML in your browser. No extra flag is needed.

Audit + exploiting confirmed SQLi

```
python hades.py -u http://testaspnet.vulnweb.com -p db_scan --exploit
```

After the audit, offers to launch sqlmap on each confirmed SQL injection (double confirmation required).

Tip: **testphp.vulnweb.com** and **testaspnet.vulnweb.com** are public test sites (Acunetix) made for practising legally.

6 · Safe mode and legality

Only audit systems you own or are explicitly authorised to test. Scanning or exploiting a database without written permission is illegal in most countries (CFAA, Computer Misuse Act, etc.).

Safe mode

In safe mode, Hades skips the 'destructive' or intrusive tests: default credentials, time-based SQL injection, and NoSQL injection. The port scan is limited to the 5 most common database ports. An INFO note tells you so.

7 · The checks explained one by one

7.1 Port scan and fingerprint

Hades tries to open a connection to known database ports (3306 MySQL, 5432 PostgreSQL, 6379 Redis, 27017 MongoDB, 9200 Elasticsearch...). For each open port it reads the service 'banner' to identify the engine and its version. If the host answers EVERY port (firewall/honeypot), the scan is skipped to avoid false positives.

7.2 Unauthenticated access + data extraction

For Redis, Memcached, Elasticsearch, CouchDB and MongoDB, Hades checks whether it can talk WITHOUT a password. If so, it does not just say it: it extracts proof — a sample of Redis keys and their count, Elasticsearch index names and document counts, the CouchDB database list. That is concrete proof an attacker could read everything. Severity: CRITICAL.

7.3 Redis CONFIG -> remote code execution (RCE)

If an open Redis also allows the CONFIG command, it is far worse than a data leak: an attacker can rewrite where Redis stores its files to drop a 'web shell', an SSH key or a scheduled task, and thus fully take over the server (RCE). Hades flags this case separately as CRITICAL.

7.4 SQL injection

Hades injects test payloads into the site's parameters (URL and forms) to see whether they reach a database query. Three techniques: via an error message, via a true/false condition (blind boolean), and via timing (a 'sleep 2 seconds' payload that slows the page measurably). Severity: CRITICAL. A ready-to-use sqlmap command is attached.

7.5 NoSQL injection

On NoSQL databases (e.g. MongoDB), Hades sends special operators (`{"$ne": null}`, `{"$gt": ""}`...) and watches whether the response changes markedly (status or size). A clear change suggests the input is treated as a query — to be verified manually. Tested outside safe mode only.

7.6 Secret files (.env, my.cnf...)

Very often the database password sits in a configuration file left readable: .env, config/database.yml, wp-config.php, my.cnf, appsettings.json, docker-compose.yml... Hades probes about thirty such paths, reads the content, and extracts the database credentials. The password is MASKED in the report. Severity: CRITICAL.

7.7 Leaked connection strings

Hades also reads the source of the site's pages and scripts looking for hard-coded connection strings (`mongodb://user:pass@...`, `postgres://...`, `jdbc:...`). A single such line can be enough to connect straight to the database. Password masked, severity CRITICAL.

7.8 GraphQL introspection

If the site exposes a GraphQL API with 'introspection' enabled, anyone can download the full schema: every query, mutation and hidden data type. Hades detects it and lists the exposed types. Severity: HIGH.

7.9 Admin interfaces

Hades looks for database management web interfaces (phpMyAdmin, Adminer, pgAdmin, CouchDB's Fauxton, Kibana, mongo-express...). An interface reachable in the clear is CRITICAL; an interface that exists but is password-protected is HIGH (remove it from the public web if not needed).

7.10 Dumps and SQLite files

Hades tries to download database backups left in the web root (backup.sql, dump.sql.gz, database.sqlite, .mdb...). A SQLite file is recognised by its signature bytes. A downloadable dump often contains the whole database — data and passwords. Severity: CRITICAL.

7.11 Framework leaks

Some frameworks expose debug pages that print the configuration, including database credentials (e.g. Spring Actuator /actuator/env, Rails). Hades probes them and reports any credential leak. Severity: CRITICAL.

7.12 TLS on database ports

Hades checks whether the database connection is encrypted (TLS). An open port without TLS means traffic — credentials included — may travel in clear text; a self-signed certificate is also flagged. Severity: LOW to MEDIUM.

7.13 Injection via HTTP headers and cookies

Developers often forget that HEADERS (User-Agent, Referer, X-Forwarded-For) and cookies sometimes end up in a SQL query (logging, analytics). So Hades injects its SQL payloads into these headers and cookies too, not only URL parameters — doubling the attack surface. Severity: CRITICAL if a SQL error comes out.

7.14 NoSQL authentication bypass

On MongoDB applications, a poorly coded login form can be tricked by sending an operator instead of a password: `{"$ne": ""}` means 'not equal to empty', i.e. 'any password'. Hades submits this kind of payload to login forms and detects whether a session opens — a full authentication bypass. Severity: CRITICAL.

7.15 Cloud databases (Firebase, Firestore, Supabase)

Many mobile/web apps use a cloud database whose address is written in the code. Hades spots it and tests whether it is publicly readable: an open Firebase Realtime Database returns ALL the data at the `/.json` address — one of the most common leaks of the modern web. Severity: CRITICAL.

8 · Understanding the output

The exposure score

At the end, Hades computes a 'DB Exposure Score' from 0 to 100 and a grade: SECURE (0-15), AT RISK (16-40), EXPOSED (41-70), CRITICAL (71-100). The higher the score, the more exposed the database. Each category of problem adds points once.

The console panel

A dedicated 'Database Security Audit' panel shows: the list of findings coloured by severity, the score bar, a summary table (ports, auth issues, injection points, data leaks, interfaces), then the two key sections below.

Loot — the extracted data

This section summarises the DATA actually retrieved during the audit: a sample of Redis keys, Elasticsearch index names, CouchDB databases, GraphQL types, leaked connection strings and secrets, default credentials. It is the 'what an attacker would walk away with' view.

Attack path — the exploitation plan

Hades turns exploitable findings into an ordered list (most to least severe) of ready-to-copy commands: the sqlmap line for each SQL injection, redis-cli for Redis, curl for Elasticsearch / CouchDB / secret files. An operator can reproduce the access step by step.

The HTML report

The HTML report (written automatically for every scan) contains a 'Database Security' section with the score gauge, the findings table, the attack path (commands in green) and the loot, plus a per-engine remediation list.

9 · Exploitation with --exploit (sqlmap)

Hades does not reinvent exploitation: it launches the real industry-standard tool, sqlmap, against confirmed SQL injections (from both the injection arsenal AND db_scan).

How it works

After the scan, if a SQLi is confirmed, Hades shows a warning panel, then asks for TWO confirmations: launch sqlmap on this parameter, and confirm you are authorised. Only then does it run sqlmap. Without those confirmations (or without `--exploit`), nothing is exploited.

Active extraction = proof of impact

With `--exploit`, db_scan no longer just reports: it EXTRACTS real data to prove impact — actual Redis key values, Elasticsearch/CouchDB documents, and the database banner via an 'in-band' SQL injection (without sqlmap). Everything is saved as EVIDENCE in the `loot/<host>_<date>/` folder.

Credential reuse

If Hades found database credentials (in a .env or a connection string), it REPLAYS them against the discovered database servers to verify whether they actually work — confirming full access. The password stays masked in the report.

Red-team report (MITRE ATT&CK;)

Each attack-path step is tagged with a MITRE ATT&CK; technique (e.g. T1190, T1078) — the standard language of security teams — and points to the matching evidence file. This turns the scan into a real engagement report.

10 · Glossary

Port: a numbered door on a server where a service listens.
Banner: text a service returns when introducing itself (name, version).
Authentication: identity verification (password, key).
SQL injection: making the database run your own commands via a site field.
NoSQL: databases without the SQL language (MongoDB, Redis...).
RCE: remote code execution = full control of the server.
Dump: a complete copy/backup of a database.
TLS: encryption of communications (the 's' in https).

11 · Remediation checklist

- Never expose a database port directly to the Internet (VPN / allowlist).
- Require a STRONG, unique password on every database (never the default values).
- Move secrets out of the web root; deny .env and config files at the server level.
- Never hard-code credentials in front-end code.
- Remove admin interfaces (phpMyAdmin, Adminer...) from public access.
- Move dumps/backups out of the web root.
- Use parameterised queries against SQL/NoSQL injection.
- Disable GraphQL introspection in production.
- Enable TLS for database connections.
- After any leak, rotate the affected password immediately.